Java DSA: Array ADT

Easy English notes from the attached Hindi YouTube lecture | Array operations, Java implementation, examples and interview revision

1. What this lecture teaches

The previous lesson practiced array questions. This lesson builds an **Array Abstract Data Type (ADT)**: a reusable Java class that stores elements and exposes common operations without requiring callers to rewrite array logic.

ADT describes what operations a data structure supports. The implementation decides how they work. The lecture uses an underlying integer array and wraps it in methods.

2. Why create an Array ADT?

Without an ADT, every caller may need to write insertion, deletion, searching, updating and traversal logic. With an ADT, callers invoke simple methods such as `insert(20)` or `search(321)`. The lecture highlights **encapsulation, reusability and abstraction**.

3. Internal state: array, capacity and element count

The lecture first discusses **array**, **size** and **capacity**, then renames its tracking variable to **index**. In the final design, the backing array has a fixed maximum capacity, while a counter tracks how many positions contain active elements.

Term	Meaning
data	Underlying int[] storing values
capacity	Maximum number of elements that fit
count / next index	Number of active elements; next insertion position

Example: capacity = 6, count = 3 means the first three positions are active and three slots remain available.

4. Operations explained step by step

Insert(value): Check whether count equals capacity. If full, reject insertion; otherwise write data[count] = value and increment count.

Delete(index): Reject a negative index or one at/after count. Shift every later active element one position left; decrease count.

Search(value): Scan active elements from index 0 to count - 1. Return the first matching index; return -1 when not found.

Get(index): Validate the requested index and return the value stored there.

Traverse(): Loop over only active elements, not the entire allocated capacity, and print each one.

Update(index, value): Validate the index, then replace the element at that position.

Get size(): Return the current count of active elements, not the allocated capacity.

5. Complete Java implementation

The following runnable example follows the lecture's fixed-capacity approach. It adds consistent index validation and clears the now-unused last slot after deletion for clarity.

```
interface ArrayADT {
    void insert(int value);
    void delete(int index);
    int search(int value);
    int get(int index);
    void update(int index, int value);
    int getSize();
    void traverse();
}

class ArrayADTImpl implements ArrayADT {
    private final int[] data;
    private int count;

    public ArrayADTImpl(int capacity) {
        if (capacity < 0) throw new IllegalArgumentException("capacity");
        data = new int[capacity];
        count = 0;
    }

    public void insert(int value) {
        if (count == data.length)
            throw new IllegalStateException("Array is full");
        data[count++] = value;
    }

    public void delete(int index) {
        checkIndex(index);
        for (int i = index; i < count - 1; i++) {
            data[i] = data[i + 1];
        }
        data[--count] = 0;
    }

    public int search(int value) {
        for (int i = 0; i < count; i++) {
            if (data[i] == value) return i;
        }
        return -1;
    }

    public int get(int index) {
        checkIndex(index);
        return data[index];
    }

    public void update(int index, int value) {
        checkIndex(index);
        data[index] = value;
    }

    public int getSize() { return count; }

    public void traverse() {
        for (int i = 0; i < count; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }

    private void checkIndex(int index) {
        if (index < 0 || index >= count)
            throw new IndexOutOfBoundsException("index: " + index);
    }
}
```

6. Using the ADT in a main method

```
public class ArrayOperations {
    public static void main(String[] args) {
        ArrayADT arr = new ArrayADTImpl(10);
        arr.insert(20);
        arr.insert(30);
```

```
        arr.insert(321);
        System.out.println("Size: " + arr.getSize()); // 3
        arr.traverse();                               // 20 30 321
        arr.update(1, 50);
        System.out.println(arr.search(321));          // 2
        arr.delete(1);
        arr.traverse();                               // 20 321
        System.out.println("Size: " + arr.getSize()); // 2
        System.out.println(arr.get(1));               // 321
    }
}
```

Save as `ArrayOperations.java`, then run `javac ArrayOperations.java` and `java ArrayOperations`.

7. How deletion works

Start with [20, 30, 321], count = 3. Deleting index 1 shifts 321 left into index 1. The active sequence becomes [20, 321], count = 2. The backing array still has its original allocated capacity; deletion changes the **logical size**.

8. Time and space complexity

Operation	Time	Extra space
Insert at end	O(1)	O(1)
Delete at index	O(n)	O(1)
Search (linear)	O(n)	O(1)
Get by index	O(1)	O(1)
Update by index	O(1)	O(1)
Traverse	O(n)	O(1)
Get size	O(1)	O(1)

Here n is the number of active elements. Constructing the backing array takes O(capacity) storage. This fixed-capacity implementation does not automatically resize.

9. Interface-based abstraction

Toward the end, the instructor extracts an **interface** describing the public operations and separates it from the concrete implementation class. A caller can depend on `ArrayADT` while the implementation is provided by `ArrayADTImpl`. Another implementation could use a different internal strategy without changing the interface.

10. Relationship to Java collections

The lecture mentions `ArrayList` and `Arrays` as built-in Java utilities. `ArrayList` is a resizable list backed by an array; `Arrays` is a utility class with static methods for arrays. They are related to this exercise but are not identical to the custom fixed-capacity ADT.

11. Interview questions and quick revision

What is an ADT? A specification of data and supported operations, independent of implementation.

What is the difference between capacity and size? Capacity is allocated room; size is the count of active elements.

Why is deletion O(n)? Later elements may need to shift left.

Why return -1 from search? It indicates that no matching element was found.

Why use an interface? To separate the operation contract from the implementation.

Can this array grow automatically? No. The lecture implements a fixed-capacity array; resizing would require extra logic.

12. Practice tasks

1. Add `isEmpty()` and `isFull()`. 2. Add `insertAt(index, value)` with right shifting. 3. Add `clear()`. 4. Create a dynamically resizing version. 5. Write tests for invalid indexes, full arrays and missing search values.

Source note: These notes summarize the attached Hindi Array ADT lecture. The complete Java example is a cleaned-up, runnable illustration based on the operations described; validation and clearing the unused slot are editorial improvements, not verbatim transcript code.